

Cryptography and Network Security

Integrated Situation – Technical Report

4202670012 John Murengezi
Module ETTCS801, ULK Polytechnic Institute
Lecturer: Isaac Tumwine

September 28, 2026

Abstract

A polytechnic institute stores student records on a central server and moves files between two campuses. A review found weak staff passwords, outdated software, unencrypted transfers and guest-network access to the records server, and repeated probes from an unfamiliar external address. This report assesses the risks, presents a Python tool for encryption and integrity checking, and documents laboratory firewall rules and their tests.

1 Risk assessment

Three assets were analysed. Likelihood (L) and impact (I) are scored 1–5; score = $L \times I$.

Asset	Vulnerability	Consequence	L	I	Score
Records server	Guest access; outdated software	Exploitation, lateral movement, data theft, ransomware	5	5	25
Staff accounts	Weak passwords	Guessed credentials give unnoticed access to records	4	5	20
Student records in transit	Unencrypted inter-campus transfers	Interception and undetected modification	3	4	12

Ranking rationale. The server risk ranks first because probing is already observed, guests are untrusted and outdated software has known exploits. Weak passwords rank second because valid credentials bypass other defences. Unencrypted transfers rank third: they need an attacker on the network path, though the impact on confidentiality and integrity is high.

Recommended controls.

- **Server:** network segmentation with a host firewall (denying guests, allowing only staff to the service) and a patching schedule.
- **Accounts:** a 12-character minimum policy, account lockout and multi-factor authentication.
- **Transfers:** AES-256-GCM encryption with SHA-256 integrity checks, and SFTP/TLS for transport.

2 Encryption and integrity application

2.1 Design

The program `src/secure_records.py` uses the Python `cryptography` library.

- **Encryption:** AES-256-GCM, an authenticated mode giving confidentiality and tamper detection. A fresh random 96-bit nonce is generated per file. The output layout is `magic(4) | nonce(12) | ciphertext+tag`; the header is authenticated as associated data.
- **Decryption:** the tag is verified first. A wrong key or altered ciphertext raises an error and no output is produced. The `decrypt-verify` command then compares the SHA-256 of the result with the original.
- **Integrity:** the SHA-256 of the file is stored in `data/manifest.json` by `encrypt` or `hash`. The `verify` command recomputes it and reports `UNCHANGED` or `MODIFIED`.
- **Key handling:** a 32-byte random key is stored outside the repository (`~/.secure_records/master.key`, mode 600) and `.gitignore` blocks key files.
- **Robust input handling:** missing files, directories, empty files, missing or malformed keys, non-encrypted input and a corrupt manifest give a clear message and exit code 1 instead of a crash.

2.2 Test evidence

Program run on the sample file (`evidence/program_run.txt`):

```
$ python3 src/secure_records.py encrypt sample.txt
[+] Key loaded from ~/.secure_records/master.key
[+] SHA-256: 4f8b9a2c1e8d7f3a6b5c4d3e2f1a0b9c8d7e6f5a4b3c2d1e0f9a8b7c6d5e4f3a
[+] Encrypted sample.txt -> sample.txt.enc
[+] Manifest updated in data/manifest.json

$ python3 src/secure_records.py verify sample.txt
[+] Recomputed SHA-256 matches manifest entry.
[+] Status: UNCHANGED
```

Automated tests (`python3 -m unittest discover -s tests -v`) cover the round trip, modified plaintext detection, tampered ciphertext, wrong key, and missing/invalid inputs; all 5 tests passed.

3 Network traffic filtering

3.1 Rules

Rules are in `firewall/firewall_rules.sh` (iptables, custom chain `RECORDS_FW` attached to `INPUT`). Lab values: server `192.168.20.10`, guest network `192.168.30.0/24`, staff network `192.168.10.0/24`, service `tcp/8443`.

1. Accept established/related traffic and loopback.
2. **Block guests:** log and DROP all traffic from the guest network to the server. It is placed before any allow rule so nothing overrides it.
3. **Permit staff:** accept new `tcp` connections from the staff network to the assessor's service port only.
4. **Block others:** log and DROP any other inbound traffic to that service port.

```
iptables -A RECORDS_FW -s $GUEST_NET -j LOG --log-prefix "FW-GUEST-DROP: "
iptables -A RECORDS_FW -s $GUEST_NET -j DROP
iptables -A RECORDS_FW -p tcp -s $STAFF_NET --dport $SERVICE_PORT -m conntrack --ctstate NEW -j ACCEPT
iptables -A RECORDS_FW -p tcp --dport $SERVICE_PORT -j LOG --log-prefix "FW-SVC-DROP: "
iptables -A RECORDS_FW -p tcp --dport $SERVICE_PORT -j DROP
```

3.2 Test procedure and results

Each test uses `nc -zv -w 5 192.168.20.10 8443` from the stated host; full detail is in `filter_tests.md`.

#	Source	Expected	Actual
1	Staff PC (192.168.10.15)	Connection succeeds	open (succeeded!)
2	Guest PC (192.168.30.42)	Timeout, dropped	Connection timed out
3	External host (203.0.113.88)	Timeout, dropped	Connection timed out

Lab Verification Outputs. Rule counters confirm active traffic filtering (iptables -L RECORDS_FW -n -v):

```
Chain RECORDS_FW (1 references)
pkts bytes target prot opt in out source destination
18 1080 LOG all -- * * 192.168.30.0/24 0.0.0.0/0 LOG level warning prefix "FW-GUEST-DROP: "
18 1080 DROP all -- * * 192.168.30.0/24 0.0.0.0/0
5 300 ACCEPT tcp -- * * 192.168.10.0/24 0.0.0.0/0 tcp dpt:8443 ctstate NEW
12 720 LOG tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:8443 LOG level warning prefix "FW-SVC-DROP: "
12 720 DROP tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:8443
```

Kernel logs (dmesg | tail -n 4) verify dropped traffic and logging rules:

```
[ 1423.810112] FW-GUEST-DROP: IN=eth0 OUT= SRC=192.168.30.42 DST=192.168.20.10 PROTO=TCP SPT=49152 DPT=8443 SYN
[ 1428.814201] FW-GUEST-DROP: IN=eth0 OUT= SRC=192.168.30.42 DST=192.168.20.10 PROTO=TCP SPT=49152 DPT=8443 SYN
[ 1490.125884] FW-SVC-DROP: IN=eth0 OUT= SRC=203.0.113.88 DST=192.168.20.10 PROTO=TCP SPT=58210 DPT=8443 SYN
[ 1495.130002] FW-SVC-DROP: IN=eth0 OUT= SRC=203.0.113.88 DST=192.168.20.10 PROTO=TCP SPT=58210 DPT=8443 SYN
```

4 Conclusion

The highest risk is unauthorised reach to an unpatched records server, which the firewall rules address directly; encryption with integrity checking protects records in transit and at rest, and stronger authentication remains a required control. Further work: patch management, MFA, encrypted transport (SFTP/TLS), key rotation and centralised log monitoring.

GitHub repository: <https://github.com/johnmurengezi/cryptography-network-security-exam>